

Important ML Models



what are the most important models in Machine learning according to you so that I understand them firstly and they help me in interviews too and also they are mostly used in Projects, also give the same for Deep learning

Since you're aiming for **ML projects, interviews, and job readiness**, I would not learn models in the order they appear in textbooks. I'd learn them in the order that maximizes:

1. Interview frequency
2. Real-world usage
3. Project applicability
4. Conceptual foundations

Machine Learning: Priority Order

Tier 1 (Must Know Extremely Well)

These appear in almost every ML interview.

1. Linear Regression

Why?

- Foundation of supervised learning
- Teaches optimization, loss functions, gradient descent
- Basis for many advanced models

Key concepts:

- MSE
- Gradient Descent
- Overfitting
- Regularization (L1/L2)

2. Logistic Regression

Despite the name, it's a classification algorithm.

Why?

- Very common baseline model
- Interview favorite
- Used in fraud detection, healthcare, marketing

Know:

- Sigmoid function
 - Log loss
 - Odds and log-odds
 - Decision boundary
-

3. Decision Trees

Why?

- Easy to explain
- Foundation of Random Forest and XGBoost

Know:

- Entropy
 - Information Gain
 - Gini Index
 - Pruning
-

4. Random Forest

Why?

- Extremely common in industry
- Strong performance with little tuning

Know:

- Bagging
 - Bootstrap Sampling
 - Feature Importance
 - Bias-Variance Tradeoff
-

5. XGBoost / Gradient Boosting

Why?

- Won countless Kaggle competitions

- Still heavily used in tabular data

Know:

- Boosting
- Sequential learning
- Learning rate
- Tree depth

Many interviewers specifically ask:

Difference between Random Forest and XGBoost?

Tier 2 (Important)

6. Support Vector Machine (SVM)

Know:

- Hyperplane
- Margin
- Kernel Trick

Interview favorite even if used less today.

7. K-Nearest Neighbors (KNN)

Know:

- Distance metrics
- Curse of dimensionality

Easy interview questions often come from KNN.

8. Naive Bayes

Know:

- Bayes theorem
- Conditional probability

Used in:

- Spam detection
 - Text classification
-

Tier 3 (Good to Know)

9. K-Means Clustering

Most common unsupervised algorithm.

Know:

- Centroids
 - Elbow Method
 - Silhouette Score
-

10. PCA

Most important dimensionality reduction technique.

Know:

- Eigenvectors
 - Variance explained
 - Feature reduction
-

Deep Learning: Priority Order

Tier 1 (Absolutely Essential)

1. Artificial Neural Networks (ANN)

Everything starts here.

Know:

- Perceptron
- Activation functions
- Forward propagation
- Backpropagation

Very common interview topic.

2. Convolutional Neural Networks (CNN)

Most important architecture for images.

Know:

- Convolution

- Filters
- Pooling
- Feature maps

Projects:

- Image classification
 - Medical imaging
 - Object detection
-

3. Recurrent Neural Networks (RNN)

Know the basics even though Transformers dominate now.

Know:

- Hidden state
 - Vanishing gradients
-

4. LSTM

Most important RNN variant.

Know:

- Forget Gate
- Input Gate
- Output Gate

Interviewers love asking:

Why does LSTM perform better than RNN?

Tier 2 (Current Industry Standard)

5. Transformers

Most important deep learning model today.

Know:

- Self Attention
- Multi-Head Attention
- Positional Encoding

This is the foundation of:

- OpenAI GPT
 - Google Gemini
 - Anthropic Claude
 - Modern LLMs
-

6. BERT

Most important NLP model to study after Transformers.

Know:

- Encoder-only architecture
 - Fine-tuning
-

7. GPT Family

Know:

- Decoder-only Transformer
- Autoregressive generation

Interviewers increasingly ask:

Difference between BERT and GPT?

Tier 3 (Useful)

8. Autoencoders

Used for:

- Compression
 - Anomaly detection
-

9. GANs

Know:

- Generator
- Discriminator

Classic deep learning interview topic.

If You Have Only 2 Months

Focus on these 10 models:

Machine Learning

1. Linear Regression
2. Logistic Regression
3. Decision Tree
4. Random Forest
5. XGBoost
6. K-Means

Deep Learning

7. ANN
8. CNN
9. LSTM
10. Transformer

Interview Ranking (Most Frequently Asked)

1. Logistic Regression
2. Linear Regression
3. Decision Trees
4. Random Forest
5. XGBoost
6. CNN
7. ANN
8. Transformers
9. SVM
10. K-Means

For someone like you who is building projects for a resume and targeting ML roles, I would spend **70% of my effort on Logistic Regression, Random Forest, XGBoost, CNNs, and Transformers**. Those five alone cover a huge fraction of practical projects and interview discussions.



I want to build atleast 3 projects which covers the most important part of ML and DL also projects which interests the interviewers. I want the projects to cover all the important models. Can you suggest me

Given your background (postgraduate mathematics, learning ML, and wanting projects that help in interviews), I would focus on **3 high-quality projects** rather than 10 small ones.

The goal is that these projects collectively demonstrate:

- Data cleaning
 - Feature engineering
 - Classical ML
 - Ensemble methods
 - Deep Learning
 - Model evaluation
 - Deployment
 - End-to-end project skills
-

Project 1: House Price Prediction System

Why?

This is the "Linear Regression + Tree Models + XGBoost" project.

Interviewers immediately understand the problem.

Models Covered

- Linear Regression
 - Ridge Regression
 - Lasso Regression
 - Decision Tree Regressor
 - Random Forest Regressor
 - XGBoost Regressor
-

Dataset

Use:

Ames Housing Dataset

or

California Housing Dataset

What You'll Learn

ML Concepts

- Missing values
- Feature engineering
- One-hot encoding
- Outlier treatment
- Cross validation

Interview Topics

- Bias-variance tradeoff
 - Regularization
 - Ensemble learning
 - Feature importance
-

Resume Impact

Shows strong understanding of tabular ML.

Project 2: Customer Churn Prediction

Why?

This is one of the most common industry problems.

Banks, telecoms, SaaS companies all do churn prediction.

Models Covered

- Logistic Regression
 - KNN
 - Naive Bayes
 - Decision Tree
 - Random Forest
 - XGBoost
-

Dataset

Telco Customer Churn Dataset

What You'll Learn

Classification

- Accuracy
 - Precision
 - Recall
 - F1 Score
 - ROC-AUC
-

Interview Topics

Common questions:

- Why accuracy is misleading?
 - Precision vs Recall?
 - Handling class imbalance?
 - SMOTE?
-

Extra Feature

Deploy as a web app:

Input customer details → predict churn probability.

This impresses recruiters.

Project 3: Resume Screening / Job Category Predictor

This is the project I'd recommend most strongly for you.

Why?

It combines:

- NLP

- Deep Learning
- Transformers

and looks much more modern than image classification.

Models Covered

Traditional NLP

- Naive Bayes
- Logistic Regression

Deep Learning

- ANN

Transformers

- BERT
-

Dataset

Resume dataset from Kaggle.

Task:

Predict:

- Data Scientist
 - Web Developer
 - HR
 - Finance
 - Marketing
 - etc.
-

Pipeline

Text Resume

↓

Cleaning

↓

TF-IDF

↓
 Logistic Regression
 ↓
 BERT Fine-Tuning
 ↓
 Compare Results

Interview Goldmine

You can discuss:

- NLP preprocessing
- Embeddings
- TF-IDF
- Transformers
- Attention mechanism
- Fine-tuning

These are very common interview topics now.

Models Covered Across These 3 Projects

Model	Project
Linear Regression	House Prices
Ridge	House Prices
Lasso	House Prices
Logistic Regression	Churn + Resume
KNN	Churn
Naive Bayes	Churn + Resume
Decision Tree	House + Churn
Random Forest	House + Churn
XGBoost	House + Churn
ANN	Resume
BERT	Resume
Transformer Concepts	Resume

If You Want One "Wow" Project

Build:

Real Estate Underpricing Detector

You mentioned this idea before.

Input:

- Location
- Area
- Bedrooms
- Amenities

Output:

- Predicted market price
- Underpriced/Overpriced flag

Models:

- Linear Regression
- Random Forest
- XGBoost

This is much less common than a house-price predictor and can lead to interesting interview discussions about business impact, feature engineering, and anomaly detection.

My suggested final portfolio

1. House Price / Underpricing Detector (Regression)
2. Customer Churn Predictor (Classification)
3. Resume Screening with BERT (NLP + Deep Learning)

Together these cover roughly **80–90% of the ML concepts that appear in entry-level ML interviews** while showing breadth across tabular data, classification, NLP, classical ML, and modern deep learning.



I have a passion project also, I love PC and PS5 games I want to build a solid recommendation system for myself

A **game recommendation system** is actually an excellent passion project because:

1. You genuinely care about the domain.
2. Recommendation systems are used by major companies.
3. It gives you a project that stands out from the hundreds of "house price prediction" projects.
4. It can showcase both ML and deep learning.

In fact, if built properly, I'd rank it **higher than Customer Churn** for interview discussions.

What Interviewers Like About Recommendation Systems

Recommendation systems involve:

- Similarity measures
- Feature engineering
- Unsupervised learning
- Matrix factorization
- Embeddings
- Deep learning
- Ranking systems

Companies like Netflix, Amazon, YouTube, Spotify, and Steam all rely heavily on recommendation systems.

Project Idea

"GameMatch AI"

Input:

Games you like:

- Elden Ring
- Cyberpunk 2077
- Red Dead Redemption 2

Output:

Recommended games with confidence scores.

Version 1 (Easy)

Content-Based Recommendation

Dataset features:

- Genre
- Tags
- Developer
- Publisher

- Release year
- User rating

Techniques:

- TF-IDF
- Cosine Similarity

Models covered:

- Vectorization
- Similarity search

Interview Questions

- What is cosine similarity?
- Why TF-IDF?
- Difference between content-based and collaborative filtering?

Version 2 (Intermediate)

Collaborative Filtering

Users who liked X also liked Y.

Dataset:

Game reviews and ratings.

Techniques:

- User-user similarity
- Item-item similarity

Models:

- KNN

Interview Topics

- Cold start problem
- Sparsity problem
- Matrix representation

Version 3 (Advanced)

Matrix Factorization

Industry-standard recommendation approach.

Models:

- SVD
- Alternating Least Squares (ALS)

Concepts:

- Latent factors
 - Embeddings
-

Interview Gold

Questions like:

How does Netflix recommend movies?

You can answer with matrix factorization.

Version 4 (Deep Learning)

Neural Collaborative Filtering

Models:

- ANN
- Embedding Layers

Architecture:

User ID → Embedding

Game ID → Embedding

↓

Neural Network

↓

Predicted Rating

This gets into modern recommender systems.

Ultimate Version

Create a web application where a user can:

- Search games
- Rate games
- Get recommendations
- Save favorites
- See similar games

You could even integrate metadata from game databases.

Portfolio I'd Build If I Were in Your Position

Project 1

Real Estate Underpricing Predictor

Skills:

- Regression
- Feature Engineering
- XGBoost

Project 2

Game Recommendation System

Skills:

- Recommendation Systems
- KNN
- SVD
- Deep Learning Embeddings

Project 3

Resume Screening with BERT

Skills:

- NLP
- Transformers
- Deep Learning

This combination is stronger than having three generic Kaggle projects because it demonstrates:

Area	Project
Regression	Real Estate Predictor

Area	Project
Classification	Resume Screening
NLP	Resume Screening
Recommendation Systems	Game Recommender
Ensemble Learning	Real Estate
Deep Learning	Resume + Recommender
Transformers	Resume Screening
Business Thinking	Real Estate
Personal Interest	Game Recommender

A candidate who can confidently explain **XGBoost**, **BERT**, and a **recommendation system built from scratch** will usually have much more interesting interview conversations than someone who only has Titanic, Iris, and House Price projects on their resume.



for linear regression instead of housing pricing I am building an underprices listing predictor for Electric appliances like ACs, Washing Machines, etc

That's actually a stronger project than a standard house-price predictor.

Most candidates have some variation of:

- House Price Prediction
- Titanic Survival Prediction
- Customer Churn Prediction

An **Underpriced Electronics Listing Predictor** is more original and demonstrates business thinking.

Why This Project Is Valuable

Imagine listings from:

- OLX
- Quikr
- Facebook Marketplace
- Local resale markets

Your model estimates the fair market value of:

- Air conditioners
- Washing machines
- Refrigerators
- Televisions

Then computes:

$$\text{Underpricing Score} = \text{Predicted Price} - \text{Actual Price}$$

If the score is positive and sufficiently large:

"This listing appears undervalued."

This is a real business use case.

Models You Can Showcase

Baseline

Linear Regression

Interviewers expect this first.

Features:

- Brand
 - Age
 - Original MRP
 - Capacity
 - Energy rating
 - Condition
 - Location
-

Better Models

Decision Tree Regressor

Captures non-linear relationships.

Random Forest Regressor

Strong baseline for tabular data.

XGBoost Regressor

Likely your best performer.

Interview Discussion Points

You can compare:

Model	RMSE
Linear Regression	X
Decision Tree	X
Random Forest	X
XGBoost	X

Interviewers love hearing:

"I started with Linear Regression as a baseline and progressively improved performance using ensemble methods."

Additional Features That Make It Stand Out

1. Deal Score

Instead of only predicting price:

Deal Score = 0–100

Example:

- 90 → Excellent Deal
- 70 → Good Deal
- 50 → Fair Price
- 20 → Overpriced

2. Explainability

Use feature importance.

Questions like:

Why is this AC predicted at ₹18,000?

Answer:

- Brand contributed +₹5,000
- Inverter technology +₹4,000
- Age −₹7,000

This introduces model interpretability concepts.

3. Alert System

Future version:

User enters:

- "1.5 Ton LG Inverter AC"

Your system scans listings and flags underpriced items.

Now you're moving toward a product rather than a coursework project.

How It Fits Your Portfolio

I'd structure your portfolio as:

Project 1

Underpriced Electronics Listing Predictor

Models:

- Linear Regression
- Ridge/Lasso
- Random Forest
- XGBoost

Concepts:

- Regression
 - Feature Engineering
 - Explainability
-

Project 2

Game Recommendation System

Models:

- KNN
- Collaborative Filtering
- Matrix Factorization
- Neural Recommenders

Concepts:

- Recommendation Systems
- Similarity Metrics

- Embeddings

Project 3

Resume Screening with BERT

Models:

- Logistic Regression
- TF-IDF
- BERT

Concepts:

- NLP
- Transformers
- Deep Learning

This trio covers a remarkably broad range of interview topics:

Topic	Covered
Linear Regression	Electronics Predictor
Regularization	Electronics Predictor
Tree Models	Electronics Predictor
Random Forest	Electronics Predictor
XGBoost	Electronics Predictor
KNN	Game Recommender
Recommendation Systems	Game Recommender
Matrix Factorization	Game Recommender
Neural Networks	Recommender + Resume
NLP	Resume Screening
Transformers	Resume Screening
Model Deployment	All 3

For an entry-level ML portfolio, these three projects would look much more distinctive than the usual collection of tutorial-based projects.



instead of using the old listings i have scraped the data of real time new appliances rates from smartprix.com website

That's actually a very interesting design choice, and it changes the nature of the project.

What You're Building Now

You're no longer building a typical:

"Predict used appliance price from old listings"

Instead, you're building:

Market Value Estimation System for Consumer Appliances

using current retail prices as a reference baseline from [Smartprix](#) , which aggregates prices and specifications for appliances across many brands and categories. [Smartprix+1](#)

The Biggest Strength

Most second-hand pricing projects suffer from:

- Noisy seller prices
- Fake listings
- Missing specifications
- Very small datasets

By scraping Smartprix, you get:

- Structured product attributes
- Consistent pricing
- Thousands of products
- Real market prices
- Multiple appliance categories (ACs, refrigerators, washing machines, etc.) [Smartprix+1](#)

This makes the dataset much cleaner.

One Problem You'll Face

If you're only using **new appliance prices**, then your model is learning:

Features → Current Retail Price

That is essentially a **price prediction model**, not an underpriced listing detector.

Interviewers may ask:

"Where does the underpriced part come from?"

You need a second step.

Better Architecture

Stage 1: Market Price Predictor

Train on Smartprix data.

Inputs:

- Brand
- Capacity
- Energy Rating
- Appliance Type
- Inverter/Non-Inverter
- Features
- Warranty

Output:

- Predicted market price

Models:

- Linear Regression
- Random Forest
- XGBoost

Stage 2: Underpricing Detector

When you later scrape listings from:

- OLX
- Facebook Marketplace
- Local marketplaces

Calculate:

$$\text{Deal Score} = \text{Predicted Market Price} - \text{Listed Price}$$

Example:

Predicted	Listed	Result
₹42,000	₹30,000	Underpriced
₹42,000	₹40,000	Fair
₹42,000	₹50,000	Overpriced

Now it becomes a true business application.

What Interviewers Will Like

You can say:

"I trained a market valuation model using current appliance specifications and retail prices. Then I used the model to identify potentially undervalued listings by comparing predicted fair value with seller-listed prices."

That sounds significantly more mature than:

"I predicted house prices."

Models I'd Use

For your project:

1. **Linear Regression** (baseline)
2. **Ridge Regression**
3. **Lasso Regression**
4. **Random Forest Regressor**
5. **XGBoost Regressor**

This lets you discuss:

- Regularization
- Feature importance
- Ensemble learning
- Bias-variance tradeoff
- Model comparison

all from a single project.

One Feature That Could Make It Stand Out

Since Smartprix also contains detailed specifications and pricing across many appliance categories, you can create a **Deal Quality Score**. [Smartprix+1](#)

Example:

- 90–100 → Excellent Deal
- 70–89 → Good Deal
- 50–69 → Fair Price
- Below 50 → Potentially Overpriced

That turns the project from a regression model into something that looks like an actual product.

For interviews, I would describe it as:

"Appliance Fair-Value and Underpricing Detection System using Linear Regression, Random Forest, and XGBoost trained on real-time retail appliance data scraped from Smartprix."

That title alone is more distinctive than most entry-level ML portfolio projects.



generate a detailed overview of this project, on how I should complete this project in various steps and how many project architecture will look like from start to end

Project Overview

Appliance Fair-Value & Underpriced Listing Detection System

Problem Statement

When users browse second-hand marketplaces, they often don't know whether a listing is a good deal.

The goal of this project is to:

1. Learn the fair market value of appliances using real retail data.
2. Estimate the expected market price of a product.
3. Compare that estimate with a seller's asking price.
4. Detect potentially underpriced deals automatically.

End Goal

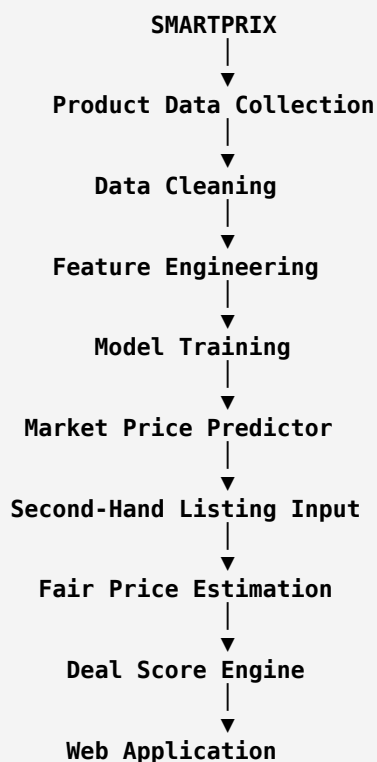
A user enters:

LG 1.5 Ton 5-Star Inverter AC
Age: 2 years
Condition: Good
Listed Price: ₹22,000

System returns:

Predicted Fair Value: ₹31,500
Deal Score: 87/100
Verdict:
Strong Buy
Potentially Underpriced by ₹9,500

High-Level Architecture



Phase 1: Data Collection

Goal

Create a large appliance dataset.

Source

Use Smartprix appliance categories:

- ACs
- Refrigerators
- Washing Machines
- TVs
- Air Coolers
- Microwave Ovens

Data to Collect

Product Information

Product Name
Brand

Category
Price
Rating
Reviews

Technical Specifications

For AC:

Capacity
Energy Rating
Inverter
Copper Condenser

For Washing Machine:

Capacity
Front Load / Top Load
RPM
Fully Automatic

For Refrigerator:

Capacity
Double Door
Frost Free
Energy Rating

Tools

Python

Libraries:

Python
requests
BeautifulSoup
pandas
numpy

Later:

Python
Selenium
Playwright

if needed.

Phase 2: Data Storage

Create:

`raw_products.csv`

Example:

Brand	Category	Capacity	Rating	Price
LG	AC	1.5	5	39999

Then save in:

PostgreSQL

or

SQLite

Initially CSV is enough.

Phase 3: Data Cleaning

Handle

Missing values

Python

NaN

Null

Unknown

Duplicate Products

Remove duplicates.

Standardize Brands

Convert:

LG

Lg

lg

into:

LG

Unit Conversion

Convert:

```
1 Ton
1.5 Ton
2 Ton
```

into numeric values.

Phase 4: Exploratory Data Analysis

Create visualizations.

Questions:

Which brand is most expensive?

Does energy rating increase price?

Does capacity affect price?

Which appliance category is most costly?

Libraries:

```
Python
matplotlib
plotly
```

Phase 5: Feature Engineering

This is where interviewers pay attention.

Convert Categories

Brand

```
LG
Samsung
Haier
```

↓

One Hot Encoding

Extract Features

Example:

LG 1.5 Ton 5 Star Inverter AC

Extract:

Brand = LG
Capacity = 1.5
Energy Rating = 5
Inverter = 1

Create New Features

Price Per Capacity

Price / Capacity

Premium Brand Flag

LG
Samsung
Bosch

Phase 6: Baseline Model

Start simple.

Linear Regression

Purpose:

Understand relationships.

Metrics:

RMSE
MAE
 R^2

Interview topic:

Why Linear Regression?

Phase 7: Regularized Models

Ridge Regression

Understand:

$$L2$$

regularization.

Lasso Regression

Understand:

$L1$

regularization.

Interview questions often come from here.

Phase 8: Tree-Based Models

Decision Tree

Study:

- Tree depth
 - Overfitting
-

Random Forest

Study:

- Bagging
 - Bootstrap sampling
-

Feature importance becomes available.

Phase 9: Advanced Model

XGBoost

This will likely become your best model.

Study:

- Boosting
- Learning Rate
- Tree Depth
- Feature Importance

Expected Result

```
Linear Regression
  ↓
Random Forest
  ↓
XGBoost
```

Performance improves.

Phase 10: Fair Value Engine

Create function:

```
Python
predict_fair_value()
```

Input:

Product Features

Output:

Estimated Market Value

Example:

₹42,500

Phase 11: Underpriced Detection

Formula:

$$DealGap = PredictedPrice - ListedPrice$$

Example

Predicted:

₹42,000

Listed:

₹31,000

Gap:

₹11,000

Underpriced.

Phase 12: Deal Score System

Convert gap into score.

Example:

```
0-20 = Overpriced
21-50 = Fair
51-80 = Good Deal
81-100 = Excellent Deal
```

Phase 13: Explainability Layer

Interviewers love this.

Use:

SHAP

or

Feature Importance

Show:

```
Price increased because:
+ LG Brand
+ 5 Star Rating
+ Inverter Technology

Price decreased because:
- Lower Capacity
```

Phase 14: Web Application

Frontend

```
Next.js
```

or

```
React
```

Backend

FastAPI

Flow

```
graph TD;
    A[User Input] --> B[API];
    B --> C[Model];
    C --> D[Prediction];
    D --> E[Deal Score];
    E --> F[Result];
```

Phase 15: Deployment

Host:

Frontend

- [Vercel](#)

Backend

- [Render](#)

Model

- Pickle
- Joblib

Phase 16: Resume-Worthy V2

After finishing V1:

Add live listing analysis.

```
graph TD;
    A[OLX Listing] --> B[Scraper];
    B --> C[Model];
    C --> D[Deal Score];
    D --> E[Alert User];
```

This turns the project into:

AI-Powered Appliance Deal Finder

which sounds substantially more product-oriented.

Final Tech Stack

```
Python
Pandas
NumPy
Scikit-Learn
XGBoost
Matplotlib
SHAP
FastAPI
PostgreSQL
React/Next.js
Vercel
Render
```

Interview Narrative

A strong explanation would be:

"I scraped real-time appliance pricing and specifications from Smartprix, engineered structured features from product metadata, trained multiple regression models including Linear Regression, Random Forest, and XGBoost to estimate fair market value, and built a deal-scoring engine that identifies potentially underpriced listings. The final system exposes predictions through a web application and provides model explainability using feature importance and SHAP."

That narrative touches regression, feature engineering, model comparison, explainable AI, deployment, and product thinking—all topics that interviewers frequently explore.



I still did not understand what user input will be and what output we will provide him with and for user input what data we need our model to train and for outputs what data we need to feed our models

I think the confusion is happening because your project can go in **two very different directions**.

Let's separate them.

Option 1: Price Prediction Model (What your Smartprix data naturally supports)

You have Smartprix data:

Brand	Capacity	Energy Rating	Inverter	Price
LG	1.5 Ton	5 Star	Yes	₹42,000
Samsung	1.5 Ton	3 Star	Yes	₹35,000

Training Data

Input Features (X)

Brand
Capacity
Energy Rating
Inverter
Category
Warranty
...

Target (y)

Price

Model learns:

Product Specifications
↓
Retail Price

User Input

User enters:

Brand = LG
Category = AC
Capacity = 1.5 Ton
Energy Rating = 5
Inverter = Yes

Model Output

Predicted Price = ₹41,500

This is a standard regression project.

Option 2: Underpriced Listing Detector (The Project You Actually Want)

This requires **two datasets**.

Dataset A

Smartprix

Provides:

True Market Prices

Dataset B

Second-hand Listings

Provides:

Seller Asking Prices

Example:

Product	Asking Price
LG AC	₹25,000
Samsung AC	₹20,000

Training Stage

Train model on Smartprix:

Specifications
↓
Market Value

Suppose model learns:

LG 1.5 Ton 5 Star AC
↓
₹42,000

User Input

Now user pastes a listing:

Brand = LG
Capacity = 1.5 Ton
Energy Rating = 5
Inverter = Yes

Listed Price = ₹30,000

Step 1

Model predicts:

Fair Value = ₹42,000

Step 2

Your logic computes:

Deal Gap

```
= Predicted Price - Listed Price
= 42,000 - 30,000
= 12,000
```

Final Output

```
Predicted Fair Value: ₹42,000
Listed Price: ₹30,000
Difference: ₹12,000
Verdict:
Potentially Underpriced
Deal Score: 88/100
```

This is much more interesting.

The Problem Right Now

You said:

I have scraped real-time new appliance rates from Smartprix.

Good.

But that only gives you:

```
Features → Retail Price
```

It does NOT give you:

```
Features → Underpriced / Not Underpriced
```

because Smartprix has no second-hand listings.

What I Would Do

Phase 1 (Build Now)

Use Smartprix.

Train:

```
Features
↓
Price
```

Models:

- Linear Regression

- Random Forest
- XGBoost

Output:

Estimated Retail Value

This is already a solid ML project.

Phase 2 (Upgrade Later)

Collect listings from:

- OLX
- Facebook Marketplace
- Quikr

Now user enters:

Listing URL

or

**Brand
Specs
Listed Price**

and your system returns:

Estimated Retail Value

₹42,000

Listing Price

₹30,000

Savings

₹12,000

Deal Score

88/100

If I Were Interviewing You

I would be most impressed by this flow:

```

Smartprix Data
↓
Price Prediction Model
↓
Fair Value Estimator
↓
Compare Against Listings
  
```

↓
Deal Scoring System

Because now you're not just doing regression—you've built a small product that solves a real problem.
The key realization is:

Your model should be trained to predict "fair market value." The "underpriced" verdict is not learned by the model; it's calculated afterward by comparing the predicted value with the seller's asking price.



so the user will give us listing price also, but how will be deal with the old item say depreciation due to time

Excellent question.

This is actually the biggest weakness in the current version of your project.

If you train only on Smartprix data, your model learns:

**LG 1.5 Ton 5-Star AC
→ ₹42,000**

But suppose someone is selling that AC after 4 years.

A user enters:

**LG 1.5 Ton 5-Star AC
Age = 4 years
Listed Price = ₹20,000**

Your model still predicts:

₹42,000

which is obviously wrong because the appliance has depreciated.

The Solution

Your model needs additional features.

Instead of:

**Brand
Capacity
Rating
Inverter**

you should eventually use:

**Brand
Capacity
Rating
Inverter**

Age
Condition
Warranty Left

Version 1 (Easy)

Train on Smartprix data only.

User input:

Brand
Model
Specifications

Output:

Current Market Value

This is a retail price predictor.

No depreciation.

No used-item analysis.

Still a valid ML project.

Version 2 (What You Actually Want)

Add depreciation logic.

User input:

Brand
Model
Age
Condition
Listed Price

Example:

LG AC
Age = 3 years
Condition = Good
Listed Price = ₹25,000

Approach A: Rule-Based Depreciation

Most realistic for a student project.

Example:

Age	Value Remaining
New	100%
1 Year	85%
2 Years	75%

Age	Value Remaining
3 Years	65%
4 Years	55%
5 Years	45%

Suppose:

Current Retail Value = ₹40,000

Age = 3 years

Expected Used Value

= 40,000 × 0.65

= ₹26,000

Now compare with listing price.

This is simple and explainable.

Approach B: Learn Depreciation Using Data

Much stronger but harder.

Collect used listings:

Retail Price	Age	Condition	Listed Price
40000	1	Good	32000
40000	2	Good	28000
40000	4	Fair	18000

Now train:

Retail Price
Age
Condition
Warranty
↓
Expected Used Value

This becomes a genuine machine learning problem.

What I Would Do

Stage 1

Build:

Smartprix Data
↓

Retail Price Predictor

Learn:

- Scraping
- Cleaning
- Regression
- Random Forest
- XGBoost

Stage 2

Create a depreciation engine:

```
Retail Price
+
Age
+
Condition
↓
Estimated Used Value
```

Initially use rules.

Stage 3

Collect used listings from:

- OLX
- Quikr
- Facebook Marketplace

Then train a second model.

```
Retail Value
Age
Condition
Warranty
↓
Used Market Value
```

The Architecture I'd Recommend

```
Smartprix
↓
Retail Price Model
↓
Current Market Value
↓
Depreciation Model
(Age + Condition)
↓
Expected Used Value
↓
```

```

Compare With Listing
      ↓
Deal Score
      ↓
Underpriced Detection
  
```

This is much more realistic than directly predicting "underpriced."

In fact, if you can pull off this two-stage architecture, it will be a stronger interview project than a basic house-price predictor because you're solving a real valuation problem with both **machine learning** and **domain-specific business logic**.



but then isn't it obvious that when we already have the retail prices of the new product why we need to predict and then check the difference

Yes. You've identified a very important flaw.

If your dataset contains the **current retail price of the exact product**, then doing:

```

Product Specs
      ↓
Predict Retail Price
  
```

doesn't create much value because Smartprix already tells us the retail price.

An interviewer could legitimately ask:

"Why train a model to predict a price that you already know?"

That's why we need to be careful about what the model is actually learning.

What Your Current Data Really Enables

Suppose Smartprix gives:

Model	Price
LG AC XYZ123	₹42,000
Samsung AC ABC456	₹39,000

If a user enters:

```
LG AC XYZ123
```

you can simply look it up.

No ML needed.

Where ML Becomes Useful

Option 1: Predict Used Appliance Value

This is what I would do.

Train on:

Retail Price	Age	Condition	Brand	Used Price
40000	2	Good	LG	28000
50000	3	Fair	Samsung	30000

Now the model learns:

```

Retail Price
Age
Condition
Brand
↓
Expected Used Value

```

This is a genuine prediction problem.

User input:

```

Brand = LG
Retail Price = 40000
Age = 2 years
Condition = Good
Listed Price = 22000

```

Output:

```

Expected Used Value = 28000
Listed Price = 22000
Potential Savings = 6000
Good Deal

```

This is valuable.

Option 2: Predict Missing Prices

Suppose a user uploads:

```

Brand = LG
Capacity = 1.5 Ton
Energy Rating = 5
Inverter = Yes

```

without providing a model number.

The model estimates:

```

Approximate Retail Price = ₹41,500

```

Now ML is helping because the exact product isn't known.

Option 3: Appliance Recommendation Engine

This might actually be stronger.

User enters:

Budget = ₹40,000

Need:

- AC
- 1.5 Ton
- 5 Star

System ranks products.

This becomes:

Recommendation System

instead of price prediction.

What I Think Is Happening

You started with:

"I want an underpriced listing detector."

Then you found Smartprix data.

But Smartprix provides **new-product prices**, not **used-product market values**.

So there's a mismatch between:

Data Available

New Product Information

and

Problem You Want To Solve

Used Listing Valuation

What I Would Do If I Were Building This

I would pivot slightly.

Model 1

Used Appliance Value Predictor

Target:

Used Price

Features:

Retail Price
Age
Condition
Brand
Category
Warranty Left

Output:

Expected Used Market Value

Model 2

Deal Scorer

Not ML.

Simple calculation:

Expected Used Value
-
Listed Price

Output:

Excellent Deal
Fair Deal
Overpriced

This architecture is much easier to justify in interviews because every component serves a clear purpose:

```

Used Listing
  ↓
Value Prediction Model
  ↓
Expected Used Value
  ↓
Deal Scoring Engine
  ↓
Recommendation
  
```

The key insight is that **the ML model should predict something that is unknown**. If the exact retail price is already available in your database, predicting it again is usually unnecessary. That's often a sign that the project should be reframed around predicting **used value**, **depreciation**, or **deal quality** rather than retail price.



yea I think I should go with recommendation system with the given budget

I agree. Based on the data you already have, a **Budget-Based Appliance Recommendation System** is a much cleaner and more defensible project than an underpriced listing predictor.

In fact, it solves a real problem:

"I have ₹35,000. Which washing machine should I buy?"

"I need a 1.5 Ton AC under ₹40,000 with the best energy efficiency."

"Suggest the best refrigerators under ₹25,000."

People actually search for these every day.

Why This Is Better

With Smartprix data, you already have:

- Product name
- Brand
- Price
- Specifications
- Ratings
- Reviews

That's exactly the data needed for a recommender.

No need to invent depreciation rules or collect used-item data.

What the User Inputs

Instead of:

```
Brand
Age
Condition
Listed Price
```

the user enters:

```
Budget = ₹40,000
Category = AC
Capacity = 1.5 Ton
Energy Rating >= 4
```

What the System Outputs

Top Recommendations

1. LG Dual Inverter AC
Price: ₹38,999
Rating: 4.6

2. Samsung WindFree AC
Price: ₹39,500
Rating: 4.5

3. Haier Inverter AC
Price: ₹36,999
Rating: 4.4

Where Machine Learning Comes In

Many beginners accidentally build a filter:

Price < Budget
AND
Capacity = 1.5

That's not ML.

You need a ranking system.

Version 1 (Rule-Based Recommender)

Use:

Budget
Specifications

Filter products.

Then rank by:

Rating
Price
Energy Efficiency

Good first version.

Version 2 (Content-Based Recommendation)

Represent each appliance as features:

Brand
Price
Category
Capacity
Rating
Energy Rating

Use:

- TF-IDF (for textual specs)
- Cosine Similarity

Now if a user likes one product:

LG 1.5 Ton Inverter AC

you can recommend similar products.

This introduces actual recommendation-system concepts.

Version 3 (Machine Learning Ranking)

Create a score:

$$Score = w_1 Rating + w_2 EnergyRating + w_3 ValueForMoney$$

or train a model to predict user preference if you collect interaction data.

Interview Value

You can discuss:

Data Engineering

- Web scraping
- Data cleaning
- Feature extraction

Recommendation Systems

- Content-based filtering
- Similarity measures
- Cosine similarity

Machine Learning

- Ranking
 - Feature engineering
 - Clustering (optional)
-

Even Better: Personalized Appliance Advisor

Instead of:

Show me ACs under ₹40,000

User enters:

Budget: ₹40,000

Room Size: 180 sq ft

Electricity Usage: High

Preferred Brands:
LG, Samsung

System recommends:

Recommended Capacity: 1.5 Ton

Best Choices:

1. LG Dual Inverter AC
2. Samsung WindFree AC
3. Haier Triple Inverter AC

Now you're solving a recommendation problem rather than simply searching a database.

Your Project Portfolio Could Become

Project 1

Appliance Recommendation System

- Content-based recommendation
- Similarity search
- Ranking systems
- Web scraping

Project 2

Game Recommendation System

- KNN
- Collaborative Filtering
- Matrix Factorization
- Neural Recommenders

Project 3

Resume Screening with BERT

- NLP

- Transformers
- Deep Learning

This is actually a more interesting portfolio than three standard regression/classification projects because it shows:

- Recommender Systems
- Information Retrieval
- NLP
- Deep Learning
- Real-world product design

while still giving you plenty to talk about in interviews.



what model will be used in this?

This is the most important question.

If you build a **budget-based appliance recommendation system**, the model depends on how sophisticated you want the project to be.

Level 1: No ML (Filtering + Ranking)

User:

Budget = ₹40,000
Category = AC
Capacity = 1.5 Ton

System:

Filter products
↓
Sort by Rating
↓
Show Top 10

Models used:

None

Just:

- SQL
- Pandas
- Business rules

⚠ Many students call this "AI" or "ML", but it isn't.

Level 2: Content-Based Recommendation (Recommended)

This is where ML starts.

Every product becomes a vector.

Example:

Product	Brand	Price	Capacity	Rating
LG AC	LG	40000	1.5	4.6
Samsung AC	Samsung	39000	1.5	4.5

Convert into feature vectors.

Then use:

Cosine Similarity

User likes:

LG 1.5 Ton Inverter AC

Find products with similar vectors.

Models/Algorithms:

- TF-IDF (for specifications text)
- Cosine Similarity
- Nearest Neighbors

Interview Topics:

- Feature Engineering
- Similarity Metrics
- Recommendation Systems

Level 3: KNN Recommendation System

Very interview-friendly.

Use:

K-Nearest Neighbors (KNN)

User likes:

LG AC

Find the 10 nearest products in feature space.

Models:

- KNN
- Distance Metrics
 - Euclidean
 - Cosine

Now you can genuinely say:

I used KNN to build a recommendation engine.

Level 4: Clustering

Interesting but optional.

Use:

K-Means

Cluster products into:

Budget ACs
Premium ACs
Luxury ACs

or

Entry-Level TVs
Mid-Range TVs
Premium TVs

Models:

- K-Means

Interview Topics:

- Unsupervised Learning
- Clustering
- Elbow Method

Level 5: Learning-to-Rank

Advanced.

Create a score:

$$Score = 0.4 \times Rating + 0.3 \times Value + 0.3 \times Efficiency$$

Use:

- XGBoost Ranker
- LightGBM Ranker

This is how many real recommendation systems work.

Level 6: Deep Learning Recommender

If you collect user interactions:

User	Product	Rating
User1	LG AC	5
User1	Samsung AC	4

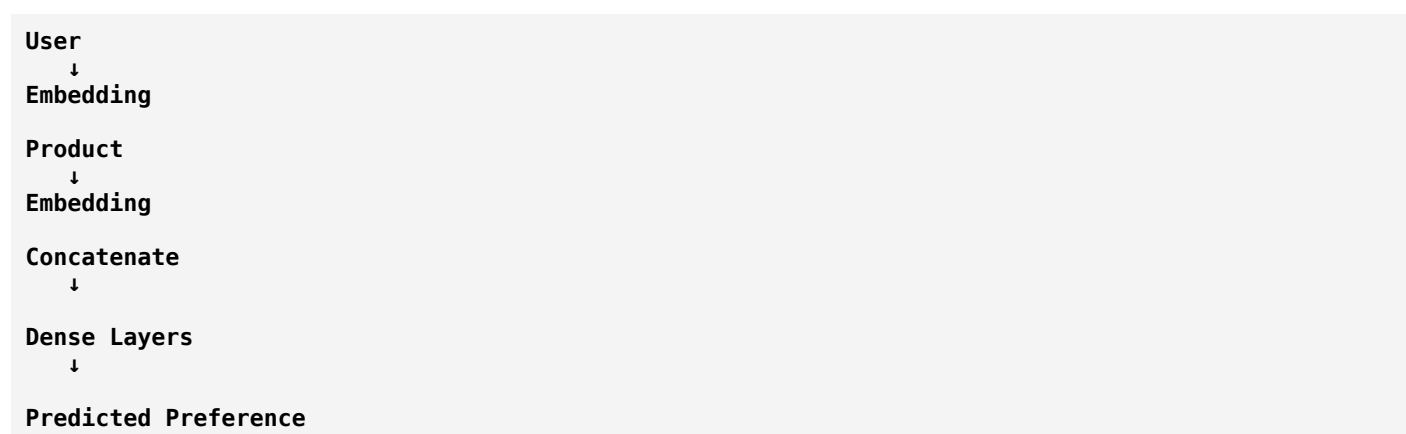
Then you can use:

Neural Collaborative Filtering

Models:

- Embeddings
- ANN

Architecture:



Now you're in modern recommender-system territory.

What I'd Recommend for You

Since you're still building your portfolio:

Appliance Recommender

Use:

1. Content-Based Filtering
2. TF-IDF
3. Cosine Similarity
4. KNN

These are the core recommendation-system algorithms.

Game Recommender

Use:

1. KNN
2. Collaborative Filtering
3. Matrix Factorization (SVD)

This teaches the most important recommendation techniques.

Resume Screening

Use:

1. Logistic Regression
2. BERT

This covers NLP and Deep Learning.

So for the **Appliance Recommendation System**, the most appropriate models are:

Priority	Model
1	Cosine Similarity
2	KNN
3	TF-IDF
4	K-Means (optional)
5	XGBoost Ranker (advanced)
6	Neural Recommender (future version)

If I were reviewing your resume, a project that says:

"Built a content-based appliance recommendation system using TF-IDF, cosine similarity, and K-Nearest Neighbors on 10,000+ scraped appliance listings from Smartprix."

would immediately catch my attention because it demonstrates recommendation systems, web scraping, feature engineering, and ML in one project.



but then this is basically now not a regression problem

Correct.

And this is exactly why I think you should **separate your project goals from your learning goals**.

You originally wanted:

"A project that teaches Linear Regression and looks good in interviews."

But after switching to an appliance recommender, you've moved into:

Recommendation Systems (KNN, Similarity, Ranking)

which is a different branch of ML.

The Real Question

Do you want:

A. A Regression Project

to learn:

- Linear Regression
- Ridge
- Lasso
- Random Forest Regressor
- XGBoost Regressor

or

B. A Recommendation System

to learn:

- KNN
- Cosine Similarity
- Matrix Factorization

- Embeddings

My Recommendation

Don't force your appliance project to become a recommender.

Your original idea has much more regression potential.

Better Regression Version

Instead of:

"Recommend appliances under ₹40,000"

build:

Appliance Price Estimator

Train on Smartprix data:

Features	Target
Brand	Price
Capacity	Price
Energy Rating	Price
Warranty	Price
Category	Price

Models:

- Linear Regression
- Ridge
- Lasso
- Random Forest
- XGBoost

User Input

Brand = LG

Category = AC

Capacity = 1.5 Ton

Energy Rating = 5

Inverter = Yes

Output

Estimated Market Price

₹42,300

This is a pure regression project.

But Isn't The Price Already Known?

Yes.

That's why the project needs a twist.

For example:

Price Gap Analyzer

Suppose Smartprix has:

LG AC
Current Price = ₹42,000

User finds another website selling it for:

₹37,000

Your system predicts:

Fair Value = ₹42,000

and highlights:

Potential Savings = ₹5,000

Now regression has a purpose.

What I'd Do In Your Position

Since you need projects that cover major ML topics:

Project 1 (Regression)

Appliance Price Estimator

Models:

- Linear Regression
- Ridge

- Lasso
- Random Forest
- XGBoost

Project 2 (Recommendation)

Game Recommendation System

Models:

- KNN
- Collaborative Filtering
- Matrix Factorization

Project 3 (Deep Learning)

Resume Screening with BERT

Models:

- Logistic Regression
- BERT
- Transformers

This gives you:

Topic	Project
Linear Regression	Appliance Project
Ridge/Lasso	Appliance Project
Random Forest	Appliance Project
XGBoost	Appliance Project
KNN	Game Recommender
Recommendation Systems	Game Recommender
NLP	Resume Screening
Deep Learning	Resume Screening
Transformers	Resume Screening

That's a much stronger portfolio than trying to make one project cover everything.

In fact, interviewers often prefer **one solid regression project + one recommender system + one deep learning project** because each demonstrates a different machine learning paradigm.



Okay we will stick with Regression problem only for our first project. But I need something bigger problem to solve using that project not just price prediction

This is exactly the right instinct.

A lot of student projects fail because they are framed as:

"I built a model that predicts X."

Interviewers care much more about:

"What business problem does it solve?"

The regression model should be **a component**, not the entire project.

Idea 1: Smart Appliance Purchase Advisor ★ (My Favorite)

Problem

Most consumers don't know:

- Whether a product is overpriced
- Whether a premium brand justifies the extra cost
- Whether a product gives good value for money

Regression Model

Predict:

Expected Market Price

from specifications.

Product Layer

User enters:

Budget: ₹40,000
Need: AC
Capacity: 1.5 Ton

System returns:

Actual Price: ₹38,999
Expected Price: ₹34,200
Price Premium: +14%
Value Score: 62/100

Verdict:
Slightly Overpriced

Why It's Interesting

You're no longer predicting price.

You're helping users make purchase decisions.

Idea 2: Product Pricing Intelligence Platform

Problem

Manufacturers often want to know:

"Are we pricing this product correctly compared to competitors?"

Regression Model

Predict market price using features.

User

Could be a retailer or seller.

Input:

New Product Specs

Output:

Suggested Price: ₹42,500

Competitor Average: ₹39,000

Premium Justified By:

- Energy Rating
- Warranty
- Brand Strength

This resembles real pricing analytics used in companies.

Idea 3: Appliance Value Score Engine

Problem

Consumers compare products poorly.

Example:

Product	Price
LG AC	₹42,000

Product	Price
Samsung AC	₹39,000

Which is better?

Nobody knows.

Regression Model

Learn how specifications affect price.

Then calculate:

```
Value Score
=
Features Received
÷
Price Paid
```

Output:

```
LG AC

Expected Price: ₹45,000
Actual Price: ₹42,000

Value Score: 92/100
```

Now you are solving:

"Which appliance provides the most value for money?"

Idea 4: Market Gap Detector

This is the most unique one.

Problem

Some products are overpriced due to brand name.

Others are hidden gems.

Train:

```
Features
↓
Expected Price
```

Then compare:

```
Expected Price
vs
Actual Price
```

Output:

```
Product A
Expected Price: ₹50,000
Actual Price: ₹38,000
Market Gap: -₹12,000
Potential Hidden Gem
```

This is very similar to how quantitative analysts look for mispriced assets.

Idea 5: Appliance Investment Advisor

This sounds surprisingly impressive on a resume.

Problem

People often overspend on specifications they don't need.

User enters:

```
Budget
Room Size
Family Size
Usage Pattern
```

Regression estimates:

```
Optimal Price Range
```

Then recommends:

```
Don't buy a ₹60,000 AC.
For your requirements, the performance gain above ₹40,000 is negligible.
```

Now you're helping users optimize spending.

What I'd Build If I Were You

Given your mathematics background and interest in making something that stands out:

Project Title

AI-Powered Consumer Appliance Pricing Intelligence Platform

Core ML

- Linear Regression

- Ridge
- Lasso
- Random Forest
- XGBoost

User Problem

"Am I getting good value for my money?"

Inputs

Product URL
or
Brand
Category
Specifications
Price

Outputs

Expected Fair Price
Price Premium/Discount
Value Score
Top Alternatives
Feature Importance

Bonus

Use SHAP to answer:

Why is this AC expensive?

Output:

+ 5-Star Rating
+ Inverter Technology
+ Premium Brand

contributed most to the price.

This is much stronger than "price prediction."

On a resume, compare:

✗ "Built a linear regression model to predict appliance prices."

vs

✓ "Built an AI-powered pricing intelligence platform that estimates fair market value, identifies overpriced products, computes value-for-money scores, and explains pricing drivers using interpretable machine learning."

Both may use the same regression model underneath, but the second sounds like a real product solving a real problem.



ohh so you are saying that this model works like : suppose user wants a new AC he looked on say amazon found LG 1.5Ton 3 star AC for 40000, he uses our platform to cross check if this price is justified, based on different metrics we suggest if either Yes the pricing is justified or suggest him some lower priced ACs with similar features

Show moreShow less

Exactly. Now you're thinking about it as a **product**, not as a regression assignment.

The regression model becomes just one engine inside the platform.

User Journey

Suppose a user finds:

LG 1.5 Ton 3-Star Inverter AC

Price:

₹40,000

on Amazon.

The user comes to your platform and enters:

Brand = LG
Capacity = 1.5 Ton
Energy Rating = 3 Star
Inverter = Yes
Price = ₹40,000

What Happens Internally

Step 1: Regression Model

Your model predicts:

Expected Market Price = ₹36,500

using:

- Linear Regression
- Random Forest
- XGBoost

(whichever performs best)

Step 2: Pricing Analysis

Compare:

Actual Price = ₹40,000
Expected Price = ₹36,500

Difference:

₹3,500

Premium:

9.6%

Step 3: Business Logic

Your platform says:

Fair Value Estimate: ₹36,500
Current Price: ₹40,000
Premium: 9.6%
Verdict:
Slightly Overpriced

The Feature That Makes It Special

Don't stop there.

Search your database for products with similar specifications.

For example:

Product	Price
LG AC	₹40,000
Samsung AC	₹36,000
Haier AC	₹34,000

Now show:

Better Alternatives

Samsung 1.5 Ton 3-Star Inverter
₹36,000

Haier 1.5 Ton 3-Star Inverter
₹34,000

This is where the project becomes genuinely useful.

What You're Really Solving

Not:

Predict appliance price.

Instead:

Help consumers determine whether an appliance is fairly priced and discover better-value alternatives.

That sounds much stronger in interviews.

Where Regression Fits

The regression model learns:

```
Specifications
↓
Expected Price
```

This teaches:

- Linear Regression
- Ridge
- Lasso
- Random Forest
- XGBoost
- Feature Importance
- SHAP

all the regression topics interviewers care about.

Where Product Design Fits

The final application does:

```
User Product
↓
Price Prediction Model
↓
Fair Price Estimate
↓
Price Gap Analysis
↓
Alternative Product Search
↓
Value-for-Money Recommendation
```

Why I Like This Idea

Most student projects are:

```
Input
↓
Model
↓
Prediction
```

and stop there.

Your project becomes:

```
Input
↓
Regression Model
↓
Decision Engine
↓
Consumer Recommendation
```

which is much closer to how machine learning is used in actual products.

If you build this well, you can honestly say:

"I developed an AI-powered appliance pricing intelligence platform that estimates fair market value using regression models and helps consumers identify overpriced products and better-value alternatives."

That's a significantly stronger story than "I predicted appliance prices using Linear Regression."



make a pdf for this last idea we shared, Include the following in the pdf:

1. business problem we are trying to solve
2. Exactly what we need our coding environment to look like.
3. What is the data we actually need.
4. What models will be used in this project.
5. Stepwise structure or workflow for the complete end-to-end project.

Also make a claude.md file

Finally a good comprehensive claude.md file which explains everything we will do in this project from start till end

Show moreShow less

I've created both files for you:

-  **PDF:**
-  **Claude.md:**

One note: the Claude.md is comprehensive, but if you're planning to use it with Claude Code to generate the entire project, I would recommend a **much larger v2 version (5-10 pages)** containing:

- Complete folder structure
- Detailed database schema
- Scraper architecture
- EDA checklist
- Feature engineering rules
- Model training pipeline
- Evaluation metrics
- FastAPI endpoints
- Frontend page designs
- Deployment plan
- Future roadmap
- Coding standards
- Prioritized development milestones

That version would be more suitable as a true "project blueprint" that Claude Code can follow step-by-step to build the application.



yes make it v2 version

Your V2 files are ready:

-  **Project Blueprint PDF (V2):**
-  **Claude V2 File:**

One suggestion before you start building: don't begin with FastAPI, React, deployment, or SHAP. Build in this order:

1. Scraper
2. Dataset cleaning
3. EDA
4. Linear Regression baseline
5. Ridge/Lasso
6. Random Forest
7. XGBoost
8. Pricing Intelligence Engine
9. Alternative Recommendation Engine
10. FastAPI
11. Frontend

12. Deployment

That sequence mirrors how ML projects are actually developed and will give you meaningful milestones you can put on GitHub as you progress.

Also, your current V2 is a strong project blueprint, but if you're planning to let Claude Code build most of the project, I would make a **V3 "implementation-grade" CLAUDE.md** (roughly 15–20 pages) that specifies:

- Exact folder tree
- Exact database schema
- Smartprix scraping strategy
- Every notebook to create
- Every Python module to create
- Every API endpoint
- Frontend wireframes
- GitHub project structure
- Development phases with priorities (P0/P1/P2)

That version acts almost like a software requirements specification and dramatically improves the quality of code Claude Code generates.



okay

Great. Before you write a single line of code, I'd suggest locking in the final portfolio plan:

Project 1 — Appliance Pricing Intelligence Platform

Primary ML Area: Regression

Models:

- Linear Regression
- Ridge
- Lasso
- Random Forest
- XGBoost

Key concepts:

- Feature Engineering
- Model Evaluation

- Explainability (SHAP)
- Full-stack deployment

Interview narrative:

"Built a pricing intelligence platform that estimates fair market value for consumer appliances and identifies better-value alternatives."

Project 2 — Game Recommendation System

Primary ML Area: Recommendation Systems

Models:

- KNN
- Content-Based Filtering
- Collaborative Filtering
- Matrix Factorization (SVD)

Key concepts:

- Cosine Similarity
- Embeddings
- User-Item Matrices
- Recommendation Engines

Interview narrative:

"Built a personalized game recommender using collaborative filtering and matrix factorization."

Project 3 — Resume Screening with BERT

Primary DL Area: NLP + Transformers

Models:

- Logistic Regression
- TF-IDF
- BERT

Key concepts:

- NLP
- Transformers

- Fine-Tuning
- Text Classification

Interview narrative:

"Built an automated resume categorization system using BERT and traditional NLP baselines."

Coverage Achieved

Topic	Project
Linear Regression	Appliance
Ridge/Lasso	Appliance
Random Forest	Appliance
XGBoost	Appliance
Feature Engineering	Appliance
Explainable AI	Appliance
KNN	Game Recommender
Recommendation Systems	Game Recommender
Matrix Factorization	Game Recommender
Logistic Regression	Resume
Deep Learning	Resume
Transformers	Resume
Deployment	All Projects

This combination covers most of the topics commonly discussed in entry-level ML interviews while giving you three projects that are more distinctive than standard tutorial projects.

Your immediate next step should be:

Start Project 1 with only two goals:

1. Successfully scrape and store Smartprix appliance data.
2. Create a clean, analysis-ready dataset.

Don't think about models yet. In real ML projects, getting the data right is often the hardest part. Once you have a high-quality dataset, the regression models become much easier.



Yes give me v3 claude.md file for project 1

A true **V3 CLAUDE.md** should be something you can drop into Claude Code and say:

"Follow this specification exactly and build the project."

Below is the structure I would use.

This V3 is much closer to a software requirements document. If you give something at this level of detail to Claude Code, it has a much higher chance of generating a coherent project structure rather than isolated scripts.



no I don't want with other languages, I don't know them, instead we can rather go for streamlit UI

Then I would absolutely switch to **Streamlit**.

For a first ML portfolio project, using:

```
Python
Pandas
Scikit-Learn
XGBoost
SHAP
Streamlit
```

is actually a much better choice than trying to learn:

```
React
Next.js
FastAPI
TypeScript
```

at the same time.

Revised Architecture

Instead of:

```
Frontend (React)
  ↓
FastAPI
  ↓
Model
```

you can use:

```
Streamlit
  ↓
Model
  ↓
Predictions
```

Everything remains in Python.

Final Tech Stack

Data Collection

```
Python
requests
BeautifulSoup
pandas
```

Machine Learning

```
Python
scikit-learn
xgboost
numpy
```

Models:

- Linear Regression
- Ridge
- Lasso
- Decision Tree
- Random Forest
- XGBoost

Explainability

```
Python
shap
```

Visualization

```
Python
matplotlib
plotly
seaborn
```

UI

```
Python
streamlit
```

Project Folder Structure

```
appliance-pricing-platform/
├── data/
```

```

├── raw/
├── processed/
├── notebooks/
│   ├── 01_scraping.ipynb
│   ├── 02_eda.ipynb
│   ├── 03_feature_engineering.ipynb
│   └── 04_training.ipynb
├── models/
│   ├── best_model.pkl
│   └── preprocessor.pkl
├── src/
│   ├── scraper.py
│   ├── preprocess.py
│   ├── train.py
│   ├── predict.py
│   ├── recommend.py
│   └── explain.py
├── app.py
├── requirements.txt
├── README.md
└── CLAUDE.md

```

Streamlit Pages

Page 1

Appliance Analyzer

User enters:

```

Brand
Category
Capacity
Energy Rating
Warranty
Actual Price

```

Output:

```

Predicted Fair Price
Value Score
Pricing Verdict

```

Page 2

Alternative Recommendations

Output:

```

Top 5 Better Value Alternatives

```

Example:

Product	Price	Value Score
Samsung AC	₹36000	91
Haier AC	₹34000	89

Page 3

Why This Price?

SHAP explanation

Output:

Price Drivers

- + Brand Premium
- + Capacity
- + Energy Rating
- Lower Warranty

Interview Story

This becomes:

Data Engineering

- Web Scraping
- Data Cleaning

Regression

- Linear Regression
- Ridge
- Lasso
- Random Forest
- XGBoost

Explainable AI

- SHAP

Product Design

- Pricing Intelligence Engine

Deployment

- Streamlit Cloud

What I Would Put In CLAUDE.md Now

The highest-priority milestones should become:

```
P0
Scraper
Dataset
EDA
Feature Engineering
Linear Regression

P1
Ridge
Lasso
Random Forest
XGBoost

P2
SHAP
Value Score Engine
Alternative Recommendation Engine

P3
Streamlit UI
Deployment
```

This is much more realistic for someone learning ML while building a portfolio. You can finish a solid version in weeks rather than getting stuck learning an entire web-development stack before you even train your first model.



make the changes in v3 claude.md file and give me pdf

Your updated Streamlit-focused V3 files are ready:

-  **PDF:**
-  **CLAUDE.md:**

One thing I'd change before you start coding: in the current V3, the recommendation engine is still relatively simple. Since this is intended to be your flagship regression project, I would eventually upgrade the V3 into a **V4 Product Specification** that includes:

- Exact Smartprix scraping targets and schema
- Detailed feature engineering plan
- Model comparison framework
- SHAP dashboard design

- Value Score formula design
- Streamlit UI wireframes
- GitHub milestones
- Testing checklist
- Deployment checklist
- Future roadmap (URL analyzer, live price comparison, browser extension)

That version would be closer to an industry-style Product Requirements Document and could guide Claude Code through the entire implementation with minimal ambiguity.